

DOCUMENTATION DU CODE ET MODÉLISATION

## Documentation Technique : Modèle Radiatif à 2 et K couches

*Équilibres radiatifs et profils de pression atmosphérique*

Ce document présente la formulation physique du profil de pression exponentiel, la loi de Beer-Lambert appliquée au transfert radiatif, ainsi que la correspondance explicite avec l'implémentation numérique Python d'un modèle à deux couches. En dernière partie on généralisera ce modèle à K couches.

### Principes physiques et Équations clés

Statique des fluides :  $\frac{dP}{dz} = -\mu g$

Gaz parfaits :  $\mu = \frac{MP}{RT_0}$

Profil de pression :  $P(z) = P_0 e^{-z/\delta}$

Beer-Lambert :  $I = I_0 e^{-\tau}$

Émissivité :  $\varepsilon = 1 - e^{-\tau}$

Stefan-Boltzmann :  $\varphi = \sigma T^4$

### Membres du groupe :

Anass, Miryana, Elvire, Cyprien, Manuel, Antoine, Rayan, Line, Merlin, Romain, Lilas,  
Victoria

# 1 Description du modèle à 2 couches

## 1.1 Données

- **Constantes fondamentales** :  $S_0 = 1361 \text{ W/m}^2$ ,  $\alpha = 0.3$ ,  $\sigma = 5.67 \times 10^{-8} \text{ W/m}^2/\text{K}^4$
- **Pression de surface** ( $P_0$ ) : 1013.25 hPa.
- **Concentration en  $\text{CO}_2$**  ( $C_{\text{CO}_2}$ ) : Fixée à 415 ppm (données NOAA).
- **Coefficient d'absorption** ( $k_{\text{CO}_2}$ ) : Paramètre effectif calibré à  $0.0035 \text{ ppm}^{-1}$ . Il représente l'opacité moyenne du  $\text{CO}_2$  dans le cadre du modèle simplifié.

## 1.2 Hypothèses du modèle

- L'axe  $z$  est orienté verticalement vers le haut, et perpendiculairement à la surface terrestre. On suppose que les couches atmosphériques s'étendent de la surface de la Terre jusqu'à  $z = +\infty$ .
- Le fluide est à l'équilibre statique dans le référentiel terrestre supposé galiléen.
- L'air est assimilé à un gaz parfait de masse molaire moyenne  $M$  constante.
- Le découpage de l'atmosphère en couches de même masse est réalisé à l'aide d'un profil de pression hydrostatique isotherme de température  $T_0$ . Une fois ce découpage effectué, le modèle radiatif attribue une température propre à la surface et à chaque couche atmosphérique. Les températures  $T_0$ ,  $T_1$  et  $T_2$  sont alors obtenues par résolution numérique des équations d'équilibre radiatif.

## 1.3 Équation d'équilibre et loi d'état

Posons  $\vec{g} = -g\vec{e}_z$ . Par la loi de la statique des fluides, on a :

$$\vec{\nabla}P = \mu\vec{g} \implies \frac{dP}{dz} = -\mu(z) \cdot g \quad (1)$$

Où  $P(z)$  désigne la pression locale et  $\mu(z)$  la masse volumique de l'air.

Sous l'hypothèse isotherme utilisée pour établir le profil hydrostatique, l'équation d'état des gaz parfaits permet d'écrire :

$$P \cdot V = \frac{m}{M}RT_0 \implies \mu(z) = \frac{M \cdot P(z)}{RT_0} \quad (2)$$

Où  $R$  est la constante des gaz parfaits.

En injectant l'expression de  $\mu(z)$  dans la relation de statique des fluides, on obtient :

$$\frac{dP}{dz} = -\left(\frac{Mg}{RT_0}\right)P(z) \quad (3)$$

On introduit la grandeur caractéristique  $\delta$  :

$$\delta = \frac{RT_0}{Mg} \quad (4)$$

Avec  $P(0) = P_0$ , on obtient le profil de pression exponentiel :

$$P(z) = P_0 \cdot e^{-\frac{z}{\delta}} \quad (5)$$

Sous l'hypothèse isotherme, la densité numérique des molécules est proportionnelle à la pression. On obtient donc :

$$n(z) = n_0 \cdot e^{-\frac{z}{\delta}} \quad (6)$$

## 1.4 Découpage de l'atmosphère en 2 couches équivalentes

Cherchons l'altitude  $z_{1/2}$  pour laquelle la pression est réduite de moitié ( $P(z_{1/2}) = \frac{P_0}{2}$ ) et qui est choisie pour séparer nos deux couches atmosphériques :

$$\frac{P_0}{2} = P_0 \cdot e^{-\frac{z_{1/2}}{\delta}} \implies e^{-\frac{z_{1/2}}{\delta}} = \frac{1}{2} \implies z_{1/2} = \delta \cdot \ln(2) \quad (7)$$

Évaluons la quantité totale de molécules par unité de surface dans la couche inférieure (de  $z = 0$  à  $z_{1/2}$ ) :

$$N_{\text{couche1}}^{2\text{couches}} = \int_0^{\delta \ln(2)} n(z) dz = n_0 \left[ -\delta \cdot e^{-\frac{z}{\delta}} \right]_0^{\delta \ln(2)} = n_0 \cdot \delta \cdot (1 - e^{-\ln(2)}) = \frac{n_0 \cdot \delta}{2} \quad (8)$$

Évaluons de même la quantité dans la couche supérieure (de  $z_{1/2}$  à  $+\infty$ ) :

$$N_{\text{couche2}}^{2\text{couches}} = \int_{\delta \ln(2)}^{+\infty} n(z) dz = n_0 \left[ -\delta \cdot e^{-\frac{z}{\delta}} \right]_{\delta \ln(2)}^{+\infty} = n_0 \cdot \delta \cdot e^{-\ln(2)} = \frac{n_0 \cdot \delta}{2} \quad (9)$$

Puisque le nombre total intégré de molécules vaut  $n_0 \cdot \delta$ , l'altitude  $z = \delta \ln(2)$  divise bien l'atmosphère en deux couches contenant exactement la même quantité de matière.

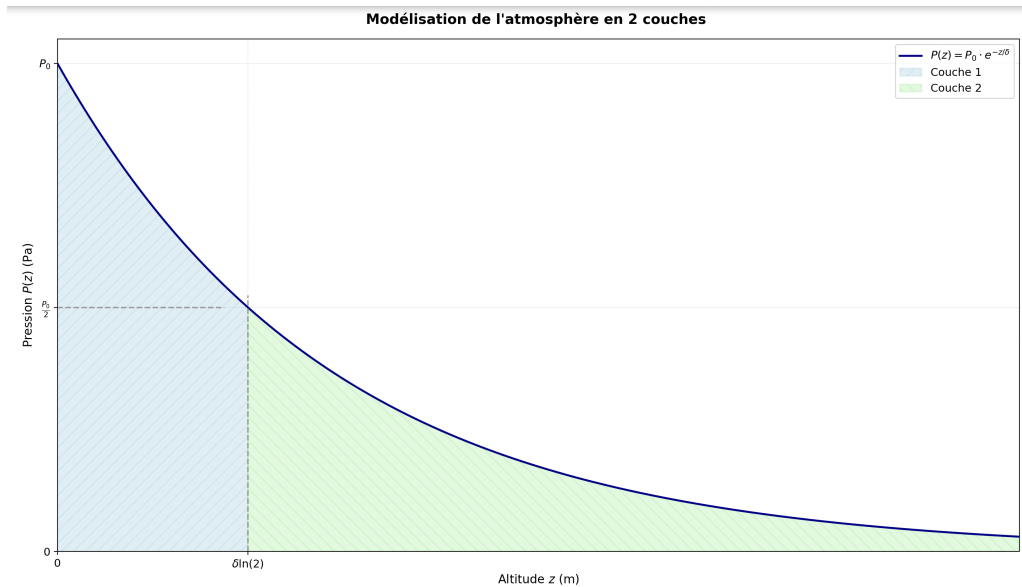


FIGURE 1 – Modélisation de l'atmosphère en 2 couches : courbe de la pression en fonction de l'altitude

**Correspondance Informatique :** L'égalité des masses implique une division égale de la pression atmosphérique totale. C'est ce qui est traduit dans le code Python lors du découpage de la masse atmosphérique :

```
1 delta_P1 = P0 / 2.0 # Couche basse (1013.25 -> 506.6 hPa)
2 delta_P2 = P0 / 2.0 # Couche haute (506.6 -> 0 hPa)
```

## 2 Transfert Radiatif : Loi de Beer-Lambert

### 2.1 Approche microscopique

On considère une tranche élémentaire d'atmosphère d'épaisseur  $dz$  et de section  $S$ , traversée par un flux infrarouge descendant.

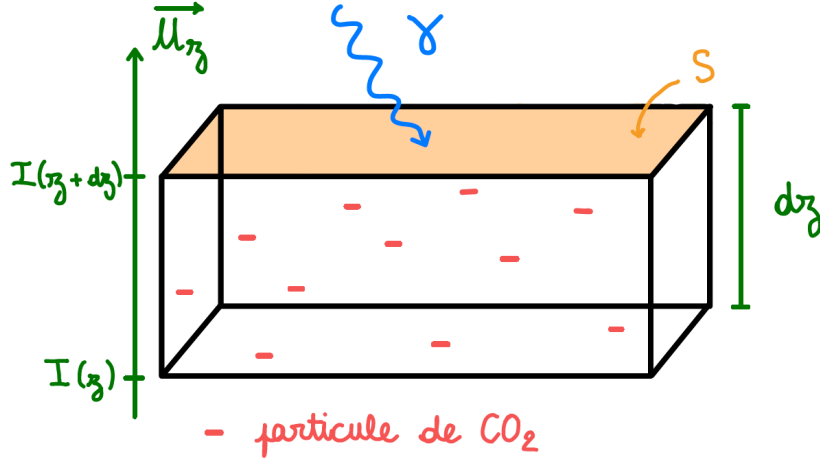


FIGURE 2 – Tranche élémentaire d’atmosphère de section  $S$  et d’épaisseur  $dz$ , traversée par un flux infrarouge descendant. L’axe  $z$  est orienté vers le haut.

- Le volume de cette tranche vaut  $dV = S \cdot dz$ .
- Soit  $n$  la densité volumique des particules de  $\text{CO}_2$ , leur nombre total dans ce volume est :  $dN = n \cdot dV = n \cdot S \cdot dz$ .
- Chaque particule de  $\text{CO}_2$  possède une section efficace d’absorption  $\sigma_{\text{abs}}$  (exprimée en  $\text{m}^2$ ). Dans ce modèle simplifié, seule l’absorption par le  $\text{CO}_2$  est prise en compte et les contributions des autres gaz atmosphériques sont négligées.
- Les molécules sont supposées n’être jamais superposées, sans quoi le flux de photons ne verrait qu’une seule section efficace effective au lieu de deux.

La surface masquée par les molécules s’écrit  $\sigma_{\text{abs}} \cdot dN$ . La probabilité  $dp$  qu’un photon soit absorbé lors de la traversée de cette couche vaut :

$$dp = \frac{\sigma_{\text{abs}} \cdot dN}{S} = \sigma_{\text{abs}} \cdot n \cdot dz \quad (10)$$

Les photons se propagent vers le bas, c’est-à-dire dans le sens des  $z$  décroissants (voir la Figure 2). L’intensité incidente pénètre ainsi dans la tranche par le haut à l’altitude  $z + dz$  et ressort atténuée par le bas à l’altitude  $z$ . L’intensité émergente s’écrit alors :

$$I(z) = I(z + dz) \cdot (1 - dp) = I(z + dz)(1 - \sigma_{\text{abs}} \cdot n \cdot dz) \quad (11)$$

En effectuant un développement limité au premier ordre, on a  $I(z + dz) \approx I(z) + \frac{dI}{dz}dz$ . En substituant cette expression et en négligeant le terme du second ordre en  $(dz)^2$ , on obtient après simplification :

$$\frac{dI}{dz} = -\sigma_{\text{abs}} \cdot n(z) \cdot I(z) \quad (12)$$

### 2.1.1 Formulation via l’épaisseur optique ( $\tau$ ) et loi de Beer-Lambert

L’épaisseur optique  $\tau$  est une grandeur sans dimension qui caractérise l’atténuation du rayonnement lors de sa propagation dans le milieu. En physique de l’atmosphère,  $\tau(z)$  est mesurée de l’altitude  $z$  jusqu’au sommet de l’atmosphère ( $z \rightarrow +\infty$ ) :

$$\tau(z) = \int_z^{+\infty} \sigma_{\text{abs}} \cdot n(z') dz' \quad (13)$$

La solution du flux s'exprime alors en fonction du flux incident au sommet de l'atmosphère  $I_\infty$  (loi de Beer-Lambert) :

$$I(z) = I_\infty e^{-\tau(z)} \quad (14)$$

Le signe négatif dans l'exponentielle traduit le fait que le rayonnement considéré se propage vers les altitudes décroissantes. Ainsi, son intensité diminue lorsqu'il traverse l'atmosphère en direction de la surface.

La quantité  $\mathcal{T} = \frac{I(z)}{I_\infty} = e^{-\tau(z)}$  représente la transmissivité de la couche atmosphérique, c'est-à-dire la fraction du rayonnement incident qui traverse la couche sans être absorbée.

En négligeant la réflexion, la conservation de l'énergie impose que la somme de l'absorptivité  $\mathcal{A}$  et de la transmissivité  $\mathcal{T}$  soit égale à 1 :

$$\mathcal{A} + \mathcal{T} = 1.$$

On en déduit donc

$$\mathcal{A} = 1 - e^{-\tau(z)}.$$

À l'équilibre thermodynamique local, la loi de Kirchhoff établit que l'émissivité  $\epsilon$  d'un milieu est égale à son absorptivité pour le rayonnement considéré. Ainsi :

$$\epsilon = \mathcal{A} = 1 - \mathcal{T} = 1 - e^{-\tau(z)}. \quad (15)$$

**Correspondance Informatique :** Dans le modèle numérique, cette relation est utilisée pour calculer l'émissivité de chaque couche à partir de son épaisseur optique. En supposant que celle-ci est proportionnelle à la concentration en  $\text{CO}_2$  et à la fraction de masse atmosphérique représentée par  $\Delta P/P_0$ , on écrit :

$$\tau = k_{\text{CO}_2} C_{\text{CO}_2} \frac{\Delta P}{P_0}, \quad (16)$$

où  $k_{\text{CO}_2}$  est un paramètre effectif calibré du modèle. L'émissivité est alors calculée par :

```
1 eps = 1.0 - np.exp(-k_CO2 * C_CO2 * (delta_P / P0))
```

## 2.2 Correspondance explicite entre le modèle physique et le code Python

La fonction `bilan_radiatif_2_couches(T)` prend en entrée un vecteur de températures

$$T = (T_0, T_1, T_2),$$

où :

- $T_0$  est la température de la surface ;
- $T_1$  est la température de la couche atmosphérique basse ;
- $T_2$  est la température de la couche atmosphérique haute.

Elle ne renvoie pas directement les températures d'équilibre, mais :

$$(\text{bilan}_{\text{surf}}, \text{bilan}_1, \text{bilan}_2).$$

L'algorithme `fsolve` ajuste alors les valeurs de  $T_0$ ,  $T_1$  et  $T_2$  jusqu'à obtenir :

$$\text{bilan}_{\text{surf}} = \text{bilan}_1 = \text{bilan}_2 = 0.$$

Le flux solaire moyen absorbé par la surface vaut :

$$\varphi_{\text{sol}} = \frac{S_0}{4}(1 - \alpha) \quad (17)$$

Le code définit ensuite les flux thermiques de corps noir associés aux trois températures :

$$\varphi_0 = \sigma T_0^4, \quad \varphi_1 = \sigma T_1^4, \quad \varphi_2 = \sigma T_2^4 \quad (18)$$

Attention :  $\varphi_1$  et  $\varphi_2$  ne sont pas encore les flux réellement émis par les couches. Ceux-ci valent respectivement  $\epsilon_1\varphi_1$  et  $\epsilon_2\varphi_2$ , puisque les couches atmosphériques ne sont pas des corps noirs parfaits.

Les trois équations codées sont alors :

### Bilan de la surface

$$\varphi_{\text{sol}} + \epsilon_1\varphi_1 + \epsilon_2\varphi_2(1 - \epsilon_1) - \varphi_0 = 0 \quad (19)$$

La surface reçoit :

- le flux solaire absorbé  $\varphi_{\text{sol}}$  ;
- le flux infrarouge descendant émis par la couche 1 :  $\epsilon_1\varphi_1$  ;
- le flux infrarouge descendant émis par la couche 2 et transmis à travers la couche 1 :  $\epsilon_2\varphi_2(1 - \epsilon_1)$ .

Elle émet :

- $\varphi_0 = \sigma T_0^4$ .

### Bilan de la couche basse

$$\epsilon_1\varphi_0 + \epsilon_1\epsilon_2\varphi_2 - 2\epsilon_1\varphi_1 = 0 \quad (20)$$

La couche 1 reçoit :

- une fraction  $\epsilon_1$  du rayonnement émis par la surface ;
- une fraction  $\epsilon_1$  du rayonnement descendant émis par la couche 2.

Elle émet :

- vers le haut :  $\epsilon_1\varphi_1$ ,
- vers le bas :  $\epsilon_1\varphi_1$ .

### Bilan de la couche haute

$$\epsilon_2\varphi_0(1 - \epsilon_1) + \epsilon_1\epsilon_2\varphi_1 - 2\epsilon_2\varphi_2 = 0 \quad (21)$$

La couche 2 reçoit :

- le rayonnement de surface transmis par la couche 1, soit  $\epsilon_2\varphi_0(1 - \epsilon_1)$  ;
- une fraction  $\epsilon_2$  du rayonnement montant émis par la couche 1, soit  $\epsilon_1\epsilon_2\varphi_1$ .

Elle émet

- vers le haut :  $\epsilon_2\varphi_2$ ,
- vers le bas :  $\epsilon_2\varphi_2$ .

Les pertes radiatives totales de la couche valent donc  $2\epsilon_2\varphi_2$ .

[SCHEMA]

## 2.3 Résolution numérique du système

Le problème consiste à déterminer les trois températures d'équilibre radiatif inconnues  $T_0, T_1, T_2$  telles que les trois bilans radiatifs soient nuls.

La fonction Python `bilan_radiatif_2_couches(T)` prend donc en entrée un vecteur de températures et renvoie les trois résidus énergétiques :

$$(\text{bilan}_{\text{surf}}, \text{bilan}_1, \text{bilan}_2).$$

La bibliothèque `scipy.optimize` fournit la fonction `fsolve`, qui cherche numériquement la racine du système non linéaire :

$$\text{bilan\_radiatif\_2\_couches}(T) = (0, 0, 0)$$

Le vecteur initial utilisé est :

```
1 T_guess = [288.0, 260.0, 230.0]
```

Il correspond à une estimation raisonnable :

- $T_0 \approx 288\text{ K}$  pour la surface ;
- $T_1 \approx 260\text{ K}$  pour la couche basse ;
- $T_2 \approx 230\text{ K}$  pour la couche haute.

La résolution numérique s'écrit :

```
1 T_equilibre = fsolve(bilan_radiatif_2_couches, T_guess)
```

La variable `T_equilibre` contient alors les températures d'équilibre radiatif obtenues pour la surface et les deux couches atmosphériques.

### 3 Généralisation du modèle à un nombre $K$ de couches

Tout le formalisme développé pour le modèle à 2 couches ne repose sur aucune hypothèse spécifique à ce nombre de couches. Il se généralise à un modèle à  $K$  couches de masse strictement identique, sans ajouter de nouvelle règle physique.

#### 3.1 Découpage vertical

Comme pour le cas à 2 couches, on divise la colonne atmosphérique en  $K$  tranches contenant toutes la même masse d'air.

On considère la pression et la quantité de matière telle que, comme démontré précédemment :

$$P(z) = P_0 e^{-z/\delta}, \quad n(z) = n_0 e^{-z/\delta}. \quad (22)$$

On cherche à découper l'atmosphère en  $K$  couches telles que chaque couche contienne la même quantité de matière.

On note  $z_q$  les altitudes de séparation avec :

$$q \in \{0, \dots, K\}, \quad z_0 = 0, \quad z_K = +\infty.$$

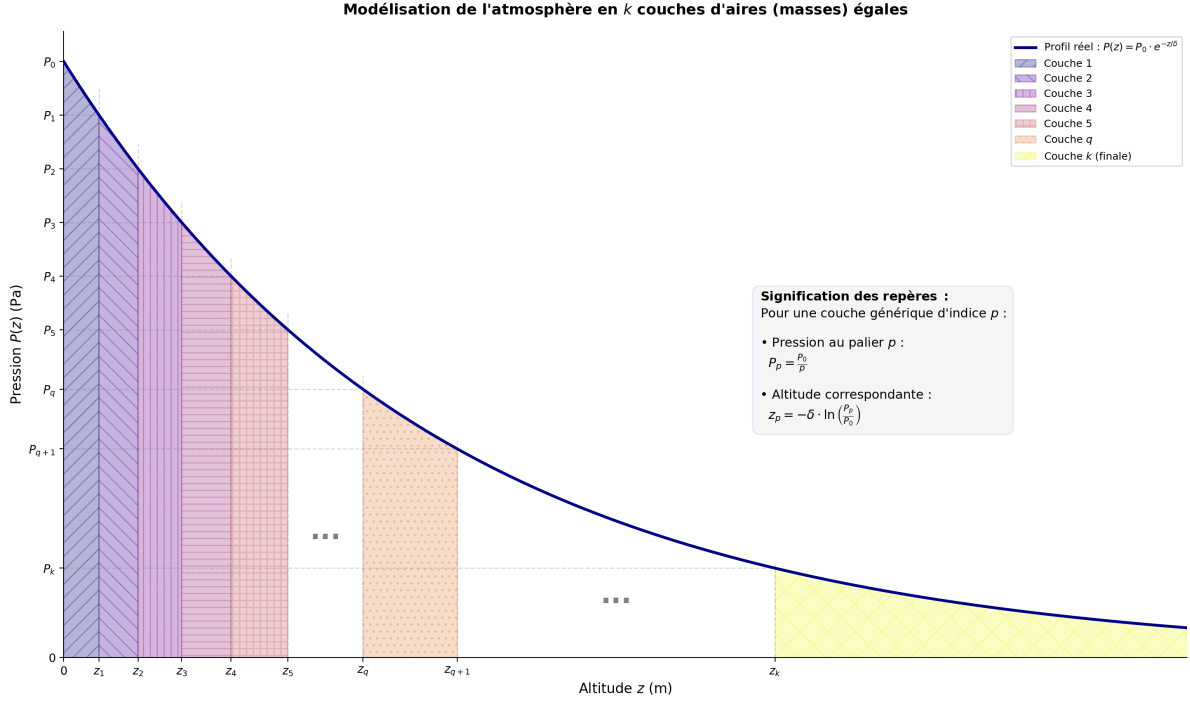


FIGURE 3 – Profil exponentiel de pression et découpage en  $k$  couches de masse identique.

La quantité de matière contenue dans la couche  $q$ , délimitée par les altitudes  $z_q$  et  $z_{q+1}$ , est donnée par :

$$N_q = \int_{z_q}^{z_{q+1}} n(z) dz. \quad (23)$$

En utilisant le profil exponentiel

$$n(z) = n_0 e^{-z/\delta},$$

on obtient :

$$N_q = \int_{z_q}^{z_{q+1}} n_0 e^{-z/\delta} dz = n_0 \left[ -\delta e^{-z/\delta} \right]_{z_q}^{z_{q+1}} = n_0 \delta \left( e^{-z_q/\delta} - e^{-z_{q+1}/\delta} \right). \quad (24)$$

La quantité totale de matière par unité de surface dans toute l'atmosphère vaut :

$$N_{\text{tot}} = \int_0^{+\infty} n(z) dz = n_0 \int_0^{+\infty} e^{-z/\delta} dz = n_0 \delta. \quad (25)$$

Si l'on découpe l'atmosphère en  $K$  couches de même masse, chaque couche doit contenir :

$$N_q = \frac{N_{\text{tot}}}{K} = \frac{n_0 \delta}{K}. \quad (26)$$

En identifiant avec l'expression précédente de  $N_q$ , on obtient :

$$e^{-z_q/\delta} - e^{-z_{q+1}/\delta} = \frac{1}{K}. \quad (27)$$

En posant  $x_q = e^{-z_q/\delta}$  la relation précédente devient  $x_q - x_{q+1} = \frac{1}{K}$ , avec la condition initiale  $x_0 = e^{-z_0/\delta} = 1$ , puisque  $z_0 = 0$ .

Ainsi, par récurrence,  $x_q = 1 - \frac{q}{K} = \frac{K-q}{K}$ , soit :

$$e^{-z_q/\delta} = \frac{K-q}{K}. \quad (28)$$



Finalement, les altitudes des interfaces sont données par :

$$\boxed{z_q = -\delta \ln \left( \frac{K - q}{K} \right)} \quad (29)$$

Il est à noter que si on repasse dans le modèle à 2 couches, l'expression reste valide.

Le profil de pression

$$P(z) = P_0 e^{-z/\delta}$$

donne alors, aux interfaces :

$$P(z_q) = P_0 e^{-z_q/\delta} = P_0 \frac{K - q}{K}. \quad (30)$$

Ainsi, la différence de pression entre deux interfaces successives vaut :

$$\boxed{P(z_q) - P(z_{q+1}) = \frac{P_0}{K}} \quad (31)$$

La figure 3 illustre ce découpage pour un nombre quelconque de couches : les couches les plus basses sont fines en altitude (car l'air est dense), les couches hautes sont plus épaisses, mais toutes contiennent la même quantité de matière.

Puisque la masse de chaque couche est identique, l'épaisseur optique et l'émissivité de toutes les couches sont identiques et se calculent exactement comme pour le modèle à 2 couches :

$$\varepsilon = 1 - \exp \left( -k_{\text{CO}_2} C_{\text{CO}_2} \frac{\Delta P}{P_0} \right) \quad (32)$$

### 3.2 Transmittance entre couches quelconques

Pour deux couches séparées par plusieurs couches intermédiaires, la fraction de rayonnement qui se transmet d'une source située à l'indice  $i$  vers une destination à l'indice  $j$  s'obtient en multipliant les facteurs de transmission  $(1 - \varepsilon)$  de toutes les couches traversées :

$$\mathcal{T}_{i \rightarrow j} = \prod_{m \text{ entre } i \text{ et } j} (1 - \varepsilon) \quad (33)$$

### 3.3 Bilans radiatifs généralisés

On note  $T_i$  la température de l'élément d'indice  $i$  (indice 0 pour la surface, indices 1 à  $k$  pour les couches atmosphériques de la plus basse à la plus haute), et  $\varphi_i = \varepsilon_i \sigma T_i^4$  le flux infrarouge émis par chaque élément (avec  $\varepsilon_0 = 1$  pour la surface qui est un corps noir). Le système de  $k + 1$  équations d'équilibre s'écrit alors :

1. *Bilan de la surface* : La surface reçoit le flux solaire et la somme du rayonnement infrarouge descendant de toutes les couches et émet son rayonnement de corps noir :

$$\frac{S_0}{4}(1 - \alpha) + \sum_{i=1}^k \varphi_i \cdot \mathcal{T}_{i \rightarrow 0} \cdot \varepsilon_0 - \varphi_0 = 0 \quad (34)$$

2. *Bilan d'une couche interne  $i$  (ni surface, ni sommet de l'atmosphère)* : La couche interne reçoit le rayonnement de tous les autres éléments, pondéré par la transmittance et sa propre émissivité. Elle émet de l'infrarouge vers le haut et vers le bas :

$$\sum_{j \neq i} \varphi_j \cdot \mathcal{T}_{j \rightarrow i} \cdot \varepsilon_i = 2\varphi_i \quad (35)$$

3. *Bilan de la couche la plus haute* : Même règle que pour les couches internes mais le rayonnement qu'elle émet vers le haut s'échappe vers l'espace.

**Correspondance avec le code :** Ce système est implémenté de manière générique dans le script :

- La fonction `calculer_transmittance(i,j)` implémente le produit des facteurs  $(1 - \varepsilon)$  entre deux indices ;
- La boucle sur les paires de couches calcule la somme des échanges radiatifs entre tous les éléments ;
- La résolution par `fsolve` fonctionne exactement comme pour 2 couches, mais avec un vecteur de températures de taille  $K + 1$  au lieu de 3.

Lorsque  $K$  augmente, le profil de température d'équilibre devient de plus en plus finement discrétisé et se rapproche d'un profil vertical continu. Dans le cadre des hypothèses du modèle, cette évolution conduit à une représentation plus réaliste de la structure thermique de l'atmosphère.

## 4 Conclusion

Ainsi, dans le cadre de ce modèle, choisir un nombre de couches élevé (par exemple  $K = 100$ ) permet d'obtenir une discrétisation suffisamment fine de l'atmosphère tout en conservant un coût de calcul raisonnable. Au-delà, les variations des températures d'équilibre deviennent négligeables, ce qui justifie ce choix numérique.

Cependant, il reste des limites au modèle comme la prise en compte simplifiée des interactions radiatives, l'absence de vapeur d'eau, de nuages et d'effets spectrales. Le modèle reste donc un cadre radiatif simplifié permettant une compréhension qualitative de l'effet de serre, sans prétendre reproduire fidèlement la physique atmosphérique réelle.

## A Code Python : Modélisation graphique du profil de pression

Dans cette annexe, nous présentons le script Python complet ayant permis de générer la figure représentant le découpage de l'atmosphère en deux couches puis en k couches de masses équivalentes.

Programme de la figure 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 #1. Constantes utiles
5 R = 8.314          # Constante universelle des gaz parfaits (J/mol/K)
6 T = 288.0          # Température moyenne (K)
7 M = 0.02896        # Masse molaire de l'air (kg/mol)
8 g = 9.81           # Accélération de la pesanteur (m/s²)
9 P0 = 101325.0       # Pression standard au niveau de la mer (Pa)
10
11 # Calcul de delta
12 delta = (R * T) / (M * g)
13
14 #2. Définition des points clés des couches
15 z_couche1 = delta * np.log(2) # Altitude où P = P0 / 2
16 P_couche1 = P0 / 2
17
18 #3. Génération des données pour la courbe
19 # On trace jusqu'à environ 4 fois delta pour bien voir l'
   # atténuation
20 z = np.linspace(0, 4 * delta, 500) # On avance 0.4*delta sur l'
   # abscisse et de 500 Pa sur l'ordonnée
21 P = P0 * np.exp(-z / delta)
22
23 #4. Création du Graphique
24 plt.figure(figsize=(9, 6))
25
26 # Tracé de la courbe principale
27 plt.plot(z, P, label=r'$P(z) = P_0 \cdot e^{-z/\delta}$', color='
   darkblue', lw=2)
28
29 # Définition des zones
30 # Zone Couche 1 : de z = 0 à z = delta*ln(2)
31 z_zone1 = np.linspace(0, z_couche1, 100)
32 P_zone1 = P0 * np.exp(-z_zone1 / delta)
33 plt.fill_between(z_zone1, 0, P_zone1, color='lightblue', alpha=0.4,
   hatch='//', label='Couche 1')
34
35 # Zone Couche 2 : à partir de z = delta*ln(2)
36 z_zone2 = np.linspace(z_couche1, 4 * delta, 300)
37 P_zone2 = P0 * np.exp(-z_zone2 / delta)
38 plt.fill_between(z_zone2, 0, P_zone2, color='lightgreen', alpha
   =0.3, hatch='\\\\\\', label='Couche 2')
39
40 # Lignes pointillées pour marquer les intersections clés
41 plt.axhline(y=P_couche1, xmax=z_couche1/(4*delta), color='gray',
   linestyle='--', alpha=0.7)
42 plt.axvline(x=z_couche1, ymax=P_couche1/P0, color='gray', linestyle
```

```

    = '--', alpha=0.7)
43
44 #5. Personnalisation des Axes et Annotations
45 # Remplacement des graduations par les valeurs th oriques du
    d bute des couches
46 plt.xticks([0, z_couche1], ['0', r'$\delta \ln(2)$'], fontsize=11)
47 plt.yticks([0, P_couche1, P0], ['0', r'$\frac{P_0}{2}$', r'$P_0$'],
    fontsize=11)
48
49 # Titres et style
50 plt.xlabel('Altitude $z$ (m)', fontsize=12)
51 plt.ylabel('Pression $P(z)$ (Pa)', fontsize=12)
52 plt.title('Mod lisation de l\'atmosphere en 2 couches ', fontsize
    =14, fontweight='bold', pad=15)
53 plt.legend(fontsize=11, loc='upper right')
54 plt.grid(True, linestyle=':', alpha=0.5)
55
56 # Ajustement des limites pour coller aux axes comme sur ton dessin
57 plt.xlim(0, 3.5 * delta)
58 plt.ylim(0, P0 * 1.05)
59
60 # Affichage du rendu
61 plt.tight_layout()
62 plt.show()

```

Programme de la figure 3

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.cm as cm
4
5 #1. Constantes Physiques
6 R = 8.314          # Constante universelle des gaz parfaits (J/mol/K)
7 T = 288.0         # Temperature moyenne (K)
8 M = 0.02896       # Masse molaire de l'air (kg/mol)
9 g = 9.81          # Accelération de la pesanteur (m/s )
10 P0 = 101325.0     # Pression standard au niveau de la mer (Pa)
11 delta = (R * T) / (M * g) # Hauteur d'echelle \delta
12
13 #2. Generation des donnees globales
14 z_max = delta * np.log(20) # On pousse pour bien voir l'asymptote
    vers 0
15 z = np.linspace(0, z_max, 1000)
16 P = P0 * np.exp(-z / delta) # Formule physique de pression
17
18 #3. Creation du Graphique
19 plt.figure(figsize=(13, 8))
20 plt.plot(z, P, color='darkblue', lw=2.5, zorder=3, label=r'Profil
    reel : $P(z) = P_0 \cdot e^{-z/\delta}$')
21
22 #4. Configuration des reperes
23
24 ticks_P = [
25     0,
26     P0 * 0.15, # Equivalent a la couche k

```

```

27     P0 * 0.35, # Equivalent a q+1
28     P0 * 0.45, # Equivalent a q
29     P0 * 0.55, # Couche 5
30     P0 * 0.64, # Couche 4
31     P0 * 0.73, # Couche 3
32     P0 * 0.82, # Couche 2
33     P0 * 0.91, # Couche 1
34     P0
35 ]
36 labels_P = [
37     '0',
38     r'$P_k$',
39     r'$P_{q+1}$',
40     r'$P_q$',
41     r'$P_5$',
42     r'$P_4$',
43     r'$P_3$',
44     r'$P_2$',
45     r'$P_1$',
46     r'$P_0$'
47 ]
48
49 # Calcul automatique des altitudes associees : z = -delta * ln(P/P0
50 )
51 ticks_z = [0]
52 for p in ticks_P[1:-1][::-1]: # Du haut vers le bas
53     ticks_z.append(-delta * np.log(p / P0))
54
55 labels_z = [
56     '0',
57     r'$z_1$',
58     r'$z_2$',
59     r'$z_3$',
60     r'$z_4$',
61     r'$z_5$',
62     r'$z_q$',
63     r'$z_{q+1}$',
64     r'$z_k$'
65 ]
66
67 # Palette de couleurs etendue
68 palette = cm.get_cmap('plasma', 8)
69
70 #5. Remplissage des couches
71 def colorer_couche_par_pression(p_sup, p_inf, couleur, hatch_style,
72     nom_label):
73     z_deb = -delta * np.log(p_sup / P0)
74     z_fin = -delta * np.log(p_inf / P0)
75     z_zone = np.linspace(z_deb, z_fin, 100)
76     P_zone = P0 * np.exp(-z_zone / delta)
77     plt.fill_between(z_zone, 0, P_zone, color=couleur, alpha=0.3,
78         hatch=hatch_style, label=nom_label)
79
80 #Couches du debut
81 colorer_couche_par_pression(P0, P0 * 0.91, palette(0), '//', '
    Couche 1')

```

```

79 colorer_couche_par_pression(P0 * 0.91, P0 * 0.82, palette(1), '\\\\',
    ', 'Couche 2')
80 colorer_couche_par_pression(P0 * 0.82, P0 * 0.73, palette(2), '||',
    'Couche 3')
81 colorer_couche_par_pression(P0 * 0.73, P0 * 0.64, palette(3), '--',
    'Couche 4')
82 colorer_couche_par_pression(P0 * 0.64, P0 * 0.55, palette(4), '++',
    'Couche 5')
83
84 # L'intervalle entre Couche 5 et Couche q reste VIDE (Premier
    espace avec '...')
85
86 # Couche q
87 colorer_couche_par_pression(P0 * 0.45, P0 * 0.35, palette(5), '..',
    'Couche $q$')
88
89 # L'intervalle entre Couche q+1 et Couche k reste VIDE (Second
    espace avec '...')
90
91 # Couche k
92 z_deb_k = -delta * np.log(0.15)
93 z_zone_k = np.linspace(z_deb_k, z_max, 300)
94 P_zone_k = P0 * np.exp(-z_zone_k / delta)
95 plt.fill_between(z_zone_k, 0, P_zone_k, color=palette(7), alpha
    =0.3, hatch='X', label='Couche $k$ (finale)')
96
97 #6. Trace des lignes pointillees de rappel
98 for i, tz in enumerate(ticks_z[1:]):
99     p_correspondante = ticks_P[:, -1][1:][i]
100     plt.axhline(y=p_correspondante, xmax=tz/z_max, color='gray',
        linestyle='--', alpha=0.3, lw=1)
101     plt.axvline(x=tz, ymax=p_correspondante/P0, color='gray',
        linestyle='--', alpha=0.3, lw=1)
102
103 #7. Placement textuel des trois petits points (...)
104 # Entre la couche 5 et la couche q
105 z_mid1 = (-delta * np.log(0.55) + -delta * np.log(0.45)) / 2
106 P_mid1 = P0 * np.exp(-z_mid1 / delta)
107 plt.text(z_mid1, P_mid1 * 0.4, '...', fontsize=24, ha='center',
    color='gray', weight='bold')
108
109 # Entre la couche q+1 et la couche k
110 z_mid2 = (-delta * np.log(0.35) + -delta * np.log(0.15)) / 2
111 P_mid2 = P0 * np.exp(-z_mid2 / delta)
112 plt.text(z_mid2, P_mid2 * 0.4, '...', fontsize=24, ha='center',
    color='gray', weight='bold')
113
114 #8. Style, Axes et Legende explicative
115 plt.xticks(ticks_z, labels_z, fontsize=10)
116 plt.yticks(ticks_P, labels_P, fontsize=10)
117
118 plt.xlabel('Altitude $z$ (m)', fontsize=12)
119 plt.ylabel('Pression $P(z)$ (Pa)', fontsize=12)
120 plt.title(r"Modelisation de l'atmosphere en $k$ couches d'aires (
    masses) egales", fontsize=13, fontweight='bold', pad=15)
121

```

```

122 # Bo te de texte explicative faisant office de legende pour la
    signification de z_p et P_p
123 texte_legende = (
124     r"$\bf{Signification\ des\ reperes\ :}$" "\n"
125     r"Pour une couche generique d'indice $p$ :" "\n\n"
126     r"     Pression au palier $p$ :" "\n"
127     r"     $P_p = \frac{P_0}{p}$" "\n\n"
128     r"     Altitude correspondante :" "\n"
129     r"     $z_p = -\delta \cdot \ln\left(\frac{P_p}{P_0}\right)$"
130 )
131 plt.text(0.62, 0.38, texte_legende, transform=plt.gca().transAxes,
    fontsize=11,
132         bbox=dict(boxstyle="round,pad=0.6", facecolor="gray",
    alpha=0.08, edgecolor="blue", lw=1))
133
134 # Legende pour les couleurs
135 plt.legend(fontsize=9, loc='upper right')
136
137 plt.xlim(0, z_max)
138 plt.ylim(0, P0 * 1.05)
139
140 plt.gca().spines['top'].set_visible(False)
141 plt.gca().spines['right'].set_visible(False)
142 plt.tight_layout()
143
144 try:
145     plt.show()
146 except KeyboardInterrupt:
147     print("\nGraphique ferme.")

```